

Algorytmy i struktury danych, Teleinformatyka, I rok

Raport z laboratorium nr: A1

Imię i nazwisko studenta: Paweł Kowalcze

nr indeksu: 420588

1. W pole poniżej wklej najważniejszy (według Ciebie) fragment kodu źródłowego z zajęć (maksymalnie 15 linii).

```
def insertionsort(arr):  
    for i in range(1, len(arr)):  
        tmp = arr[i]  
        j = i - 1  
        while j >= 0 and arr[j] > tmp:  
            arr[j + 1] = arr[j]  
            j = j - 1  
        arr[j + 1] = tmp
```

Uzasadnij swój wybór.

Ten fragment był dla mnie najważniejszy, ponieważ pomógł mi on zrozumieć złożoność obliczeniową algorytmów. W przypadku sortowania przez wstawianie, złożoność obliczeniowa zależy od tego jaka lista jest sortowana i wynosi n dla najlepszego przypadku (czyli już posortowanych list) i n^2 dla najgorszego przypadku (czyli listy posortowanej w odwrotnej kolejności).

2. Podsumuj wyniki uzyskane podczas wykonywania ćwiczenia. Co ciekawego zauważyłeś? Czego się nauczyłeś? Jeśli instrukcja zawierała pytania, odpowiedz na nie. Do sprawozdania możesz dodać wykresy, tabele jeśli jest taka potrzeba.

Czas wykonania algorytmów dla tablic posiadających 1000 elementów:

Mergesort:

Średni czas 0.0023736913204193116

Najwolniejszy czas: 0.003187894821166992

Najszybszy czas: 0.0019638538360595703

Całkowity czas wykonania funkcji mergesort: 2.3916618824005127

Insertionsort:

Średni czas 0.02090549945831299

Najwolniejszy czas: 0.024333715438842773

Najszybszy czas: 0.018996477127075195

Całkowity czas wykonania funkcji insertionsort: 20.924638271331787

Czas wykonania algorytmów dla tablic posiadających 100 elementów:

Mergesort:

Średni czas 0.0011093480587005616

Najwolniejszy czas: 0.0029997825622558594

Najszybszy czas: 0.00012087821960449219

Całkowity czas wykonania funkcji mergesort: 1.128516435623169

Insertionsort:

Średni czas 0.00496699595451355

Najwolniejszy czas: 0.006228923797607422

Najszybszy czas: 0.003966808319091797

Całkowity czas wykonania funkcji insertionsort: 4.984001636505127

Różnica czasu działania obu algorytmów zmienia się wraz z ilością danych do posortowania - im więcej danych tym dłuższy czas. Złożoność obliczeniowa funkcji

mergesort wynosi $n \cdot \log(n)$ a insertionsort n^2 . Potwierdzają to zmierzone czasy, gdyż przy 10 krotnym zwiększeniu ilości elementów, czas mergesortu zwiększył się nieznacznie a insertionsortu jest to znaczna różnica. Mergesort ma większą złożoność pamięciową.

Na tych zajęciach nauczyłem się co to jest złożoność obliczeniowa algorytmu, jak ona działa i jak wpływa na czas wykonywania się programów. Teraz wiem, że jest to bardzo istotna kwestia, i trzeba na to zwracać uwagę przy pisaniu programów.